Introduction to Collections

08 February 2025 20:19

An array is an indexed collection of fixed number Homogenous data elements.
The main Advantage of arrays is, we can represent multiple values, by using single variable. So, that readability of the code will be improved.

Limitations of Arrays:

1. Array are fixed in size, i.e. once we create an array there is no chance of increasing or decreasing the size based on our requirement. Due to this, to use array concept we should know the size in advance which may not be possible always.
2. Array can hold homogeneous data type elements.

   ```
   Student[] s = new Student[10000];
   s[0] = new Student();
   s[1] = new Customer(); //C.E.: incompatible types found Customer, required: Student
   ```

   We can solve this problem using Object type arrays.

   ```
   Object[] a = new Object[10000];
   a[0] = new Student();
   a[1] = new Customer();
   ```

   Array concept is not implemented based on some standard data structure and hence ready-made methods support is not available. For every requirement, we have to write the code explicitly, which increases complexity of programming.

   To overcome above problems of arrays, we should go for **Collections concept**.

1. Collections are global in nature, that is based on our requirement. We can increase or decrease the size.
2. Collection can hold both heterogeneous and homogeneous objects.
3. Every collection class is implemented based on some standard data structure. Hence, for every requirement, ready-made method support is available.
4. Being a programmer, we are responsible to use those methods. We are not responsible to implement those methods.

Difference between Arrays and Collections

Arrays	**Collections**
Fixed in size that is. once we create an array, we cannot increase or decrease the size based on our requirement.	Growable in nature that is based on our requirement. We can increase or decrease the size.
With respect to memory, arrays are not recommended to use.	With respect to memory collections are recommended to use.
With respect to performance arrays are recommended to use.	With respect to performance collections are not recommended to use.
Arrays can hold only homogeneous data type elements.	Collections can hold both homogeneous and heterogeneous elements.
No underlying data structure for arrays. And hence ready-made methods support is not available for. every requirement we have to write the code explicitly, which increases complexity of programming.	Every collection class is implemented based on some standard data structure. And hence, for every requirement, ready-made method support is available Being a programmer, we can use these methods directly. And we are not responsible for. implementing those methods.
Arrays can hold both primitive and object types.	Collection can hold only object types, but not primitives.

```
int x = 10;

Integer y = x;
```

wrapping
or
//boxing: mean primitive type is converted to object.

y → value 10

The advantage of boxing is, we can use the respected class methods on the given data.

Unwrapping
or

int m = y; //Unboxing

```
import java.util.Scanner;
public class WrapperDemo {
    public static void main(String[] args) {
        int x;
        Scanner sc = new Scanner(System.in);
        System.out.println("\fEnter integer value for x = ");
        x = sc.nextInt();
        Integer y = x; //boxing
        System.out.println("Hexadecimal equivalent: " + Integer.toHexString(y));
        System.out.println("Binary equivalent: " + Integer.toBinaryString(y));
    }
}
```

What is Collection?

If we want to represent a group of individual object as a single entity then we should go for Collection.

Collection is a Framework(API). It contains several classes and Interfaces which can be used to represent a group of individual objects a single entity.

Java	C++
Collection	Container
Collection Framework	STL(Standard Template library)

<u>9 Key interfaces of Collection Framework</u>

1. Collections
2. List
3. Set
4. SortedSet
5. NavigableSet
6. Queue
7. Map
8. SortedMap
9. NavigableMap

Collection(I): If we want to represent a group of individual object as a single entity, then we should go for collection. Collection interface defines the most common methods which are applicable for any collection object. In general, collection interface is considered as root interface of collection framework. There is no concrete class which implements collection interface directly.

Collection interface defines the most common methods which are applicable for any collection object. In general, collection interface is considered as root interface of collection framework. There is no concrete class which implements collection interface directly.

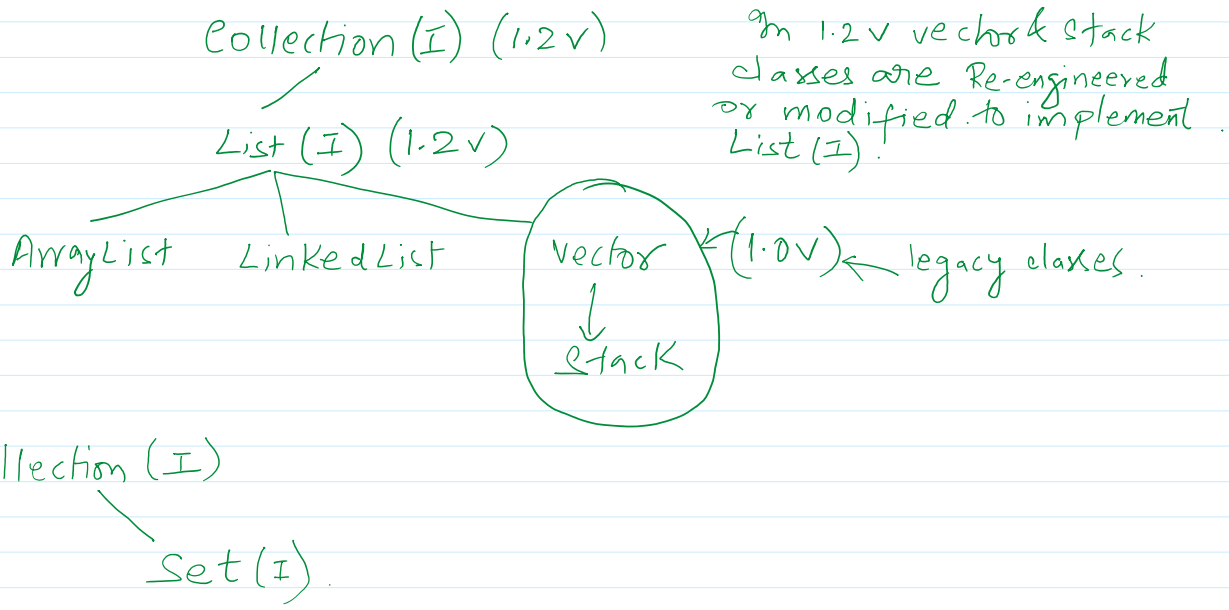
C = class.
I = Interface.

What is the difference between Collection(I) and Collections?

Collections class is an utility class present in java.utill package, to define several utility methods for Collection interface objects (Like sorting, searching, etc.)

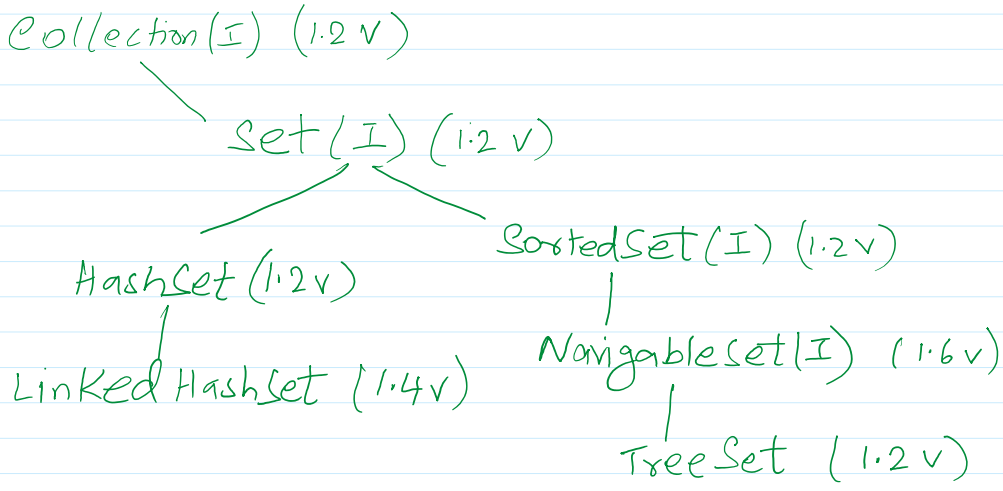
Collection (I)
List (I)

List interface: It is a child interface of **Collection Interface**. If we want to represent a group of individual objects that has a single entity where duplicates are allowed and insertion order must be preserved then we should go for **List interface.**



Set Interface:

- **i.** It is a child interface of **Collection** interface.
- **ii.** If we want to represent a group of individual objects as a single entity where duplicates are not allowed and insertion order not required to preserve, then we should go for set interface.



SortedSet Interface:

i. It is a child interface of **Set** interface.
ii. If we want to represent a group of individual object as a single entity where duplicates are not allowed and all objects should be inserted according to some sorting order, then we should go for **SortedSet** interface.

NavigableSet interface:

i. It is a child interface of **SortedSet** interface.
ii. It contains several methods for navigation purpose (navigable).

What is the difference between set interface and list interface?

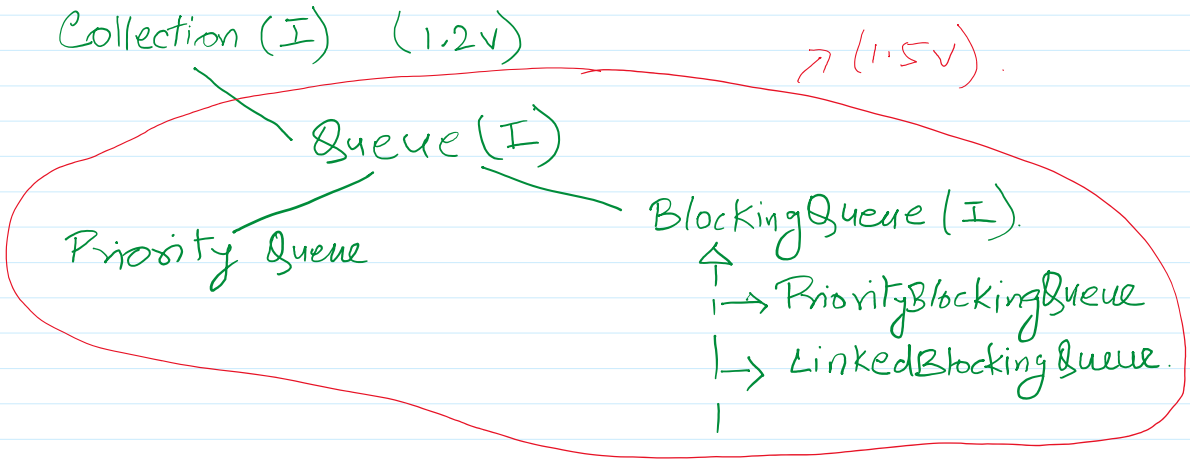
Set interface	List interface
Duplicates are not allowed.	Duplicates are allowed.
Insertion order is not preserved.	Insertion order is preserved.

collection(I) "# prior to processing"

Queue(I) (FIFO)

Queue interface:

i. Queue is a child interface of **Collection** interface.
ii. If we want to represent a group of individual **"prior to processing"** objects, then we should go for **Queue** Interface.
iii. Usually Queue follows (FIFO) Order but based on our requirement, we can implement our own priority order also. For example, before sending a mail, all mail-ids, we have to store in some data structure, in which order we added mail-ids, in the same order mail should be delivered, for this requirement Queue is the best choice.

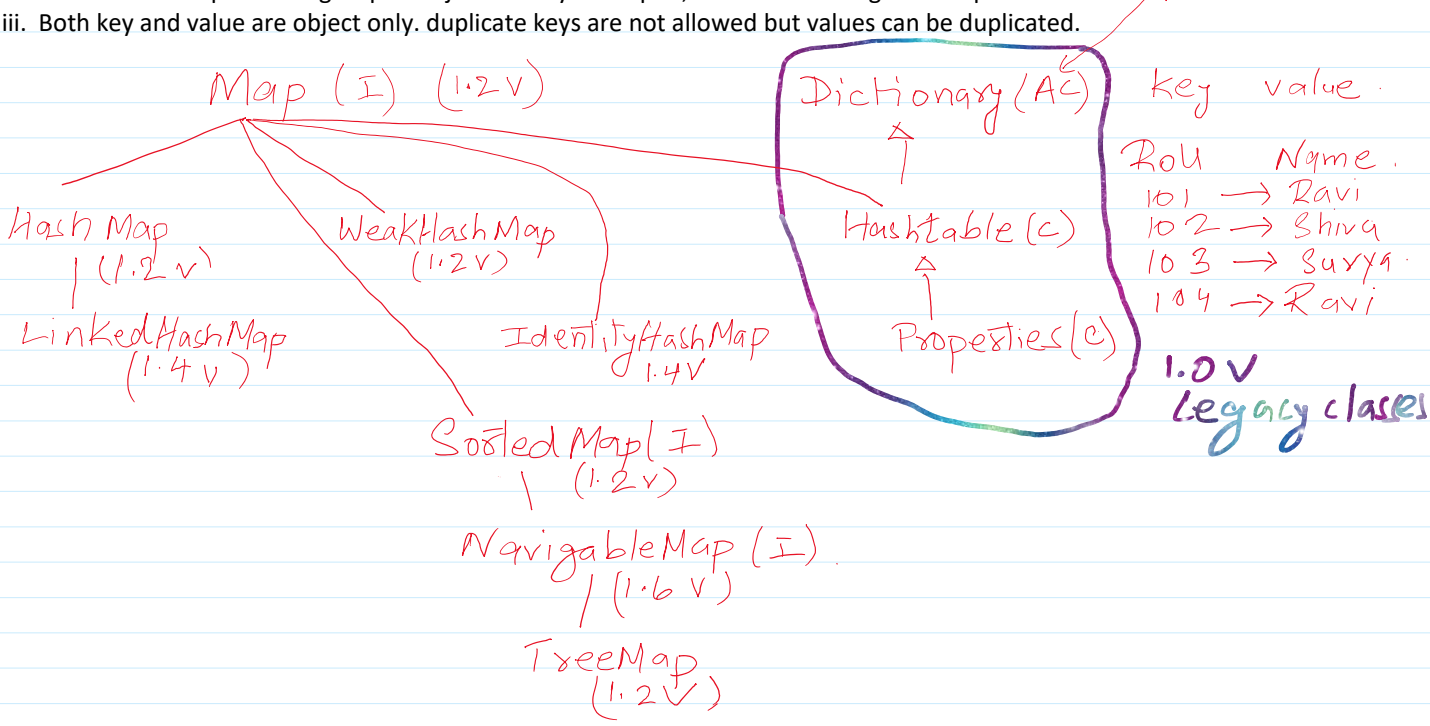


Note: All the above interface (Collection, List, Set, SortedSet, NavigableSet, Queue) meant for representing a group of individual objects.

If we want to represent a group of object as key value pair, then we should go for Map interface.

Map Interface:

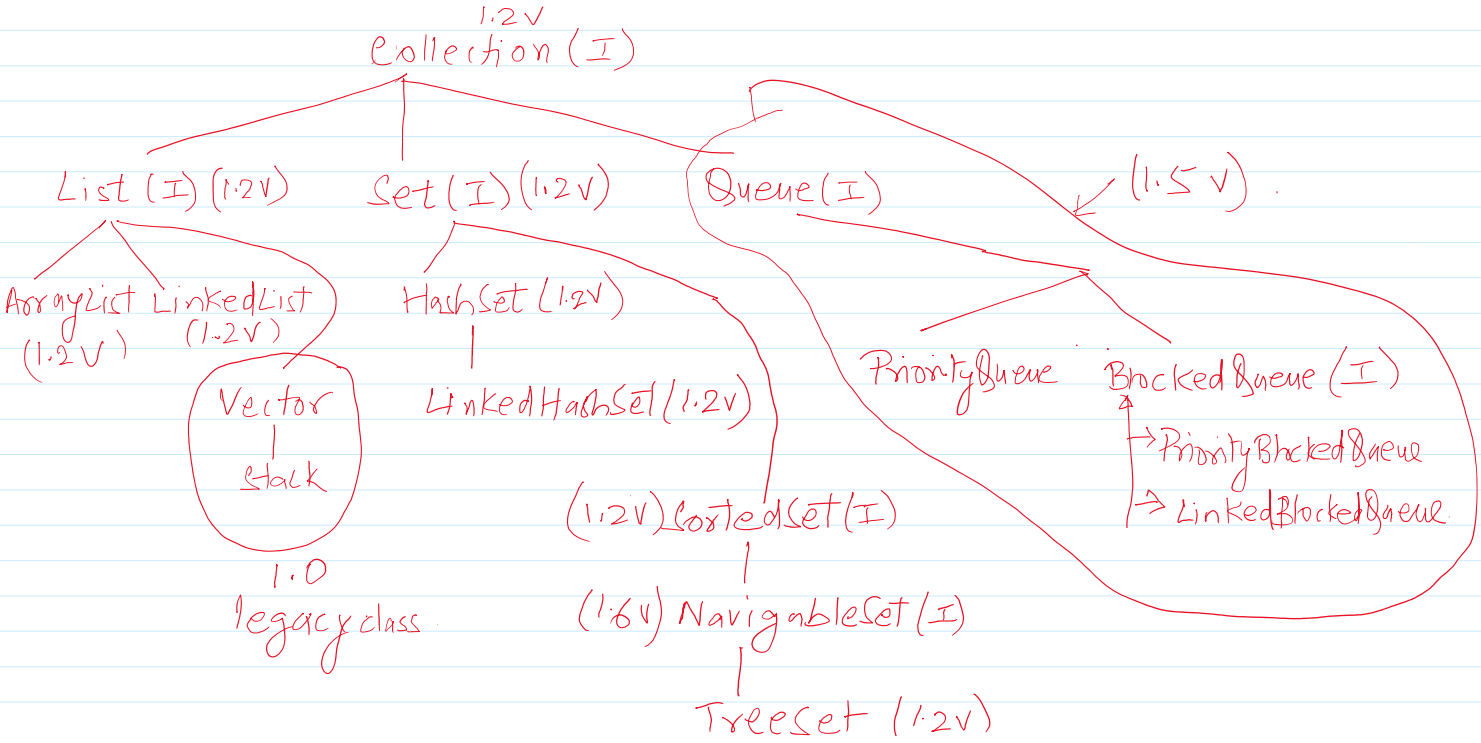
i. Map interface is not a child interface of Collection interface.
ii. If we want to represent a group of objects as key value pair, then we should go for map interface.
iii. Both key and value are object only. duplicate keys are not allowed but values can be duplicated.



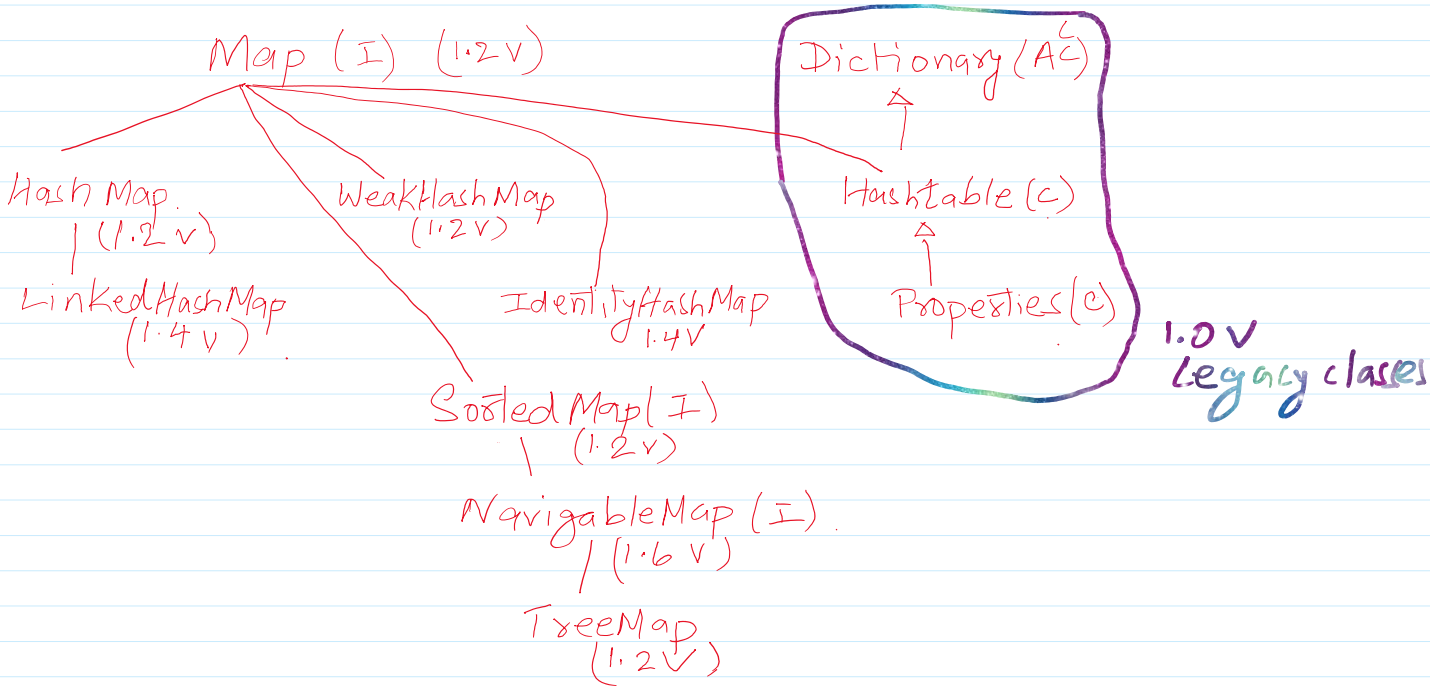
SortedMap Interface:

i. It is a child interface of Map interface.
ii. If we want to represent a group of key-value pairs according to some sorting order of keys, then we should go for sorted map interface.

NavigableMap interface: It is a child interface of **SortedMap** interface. It defines several methods for navigation purpose.



Treeset (1.2v)



Sorting

1. Comparable (I) :- Default natural sorting order (Ascending order).
2. Comparator (I) :- customized sorting order.

Cursors (get objects one by one).

1. Enumeration (I). 1.0v
2. Iterator (I).
3. ListIterator (I).

Utility classes.

Collections Arrays.

This is total collection framework.

Legacy classes, and Interfaces in Collection framework.

→ Enumeration (I)

Legacy classes, and Interfaces in Collection framework.

- ⇒ Enumeration (I)
- ⇒ Dictionary (Abstract class).
- ⇒ Vector
- ⇒ Stack.
- ⇒ Hash table.
- ⇒ Properties.